
AiM Simulator Summary

***Intelligent System
Laboratory***

AiM Simulator CMDs

❑ Deep Learning CMDs

- Simulator의 CMD와 JSSCC의 CMD는 구성이 약간 다르다
- Simulator의 CMDs

TABLE I NEWLY DEFINED DEDICATED DL CMD SET		
Type	CMD	Description
Bank Activation	ACT4, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks parallel
Compute	MACSB, MAC4B, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWML	Perform element-wise multiplication
Data	RDCP	Copy data from a bank to global buffer
	WRCP	Copy data from global buffer to a bank
	WRGB	Write to global buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

[GDDR6 based AiM paper의 Table]

유형	명령어	세부 사항
AiM	WR_SBK [GPR_0] [channel_mask] [bank] [row]	GPR에서 256b를 Single Bank로 Write
AiM	WR_ABK [GPR_0] [channel_mask] [row]	GPR에서 256b를 16개 Bank로 Write
AiM	WR_GB [opsize] [GPR_0] [channel_mask]	GPR에서 256b를 GB로 Write
AiM	WR_BIAS [GPR_0] [channel_mask]	GPR에서 256b를 16개 Bank의 16b MAC Reg로 Write
AiM	RD_AF [GPR_0] [channel_mask]	16개 Bank의 16b AF Reg에서 256b를 GPR로 Read
AiM	RD_SBK [GPR_0] [channel_mask] [bank] [row]	Single Bank에서 256b를 GPR로 Write
AiM	COPY_BKGB [opsize] [channel_mask] [bank] [row]	16개 Bank에서 256b를 GB로 Copy
AiM	COPY_GBBK [opsize] [channel_mask] [bank] [row]	GB에서 256b를 16개 Bank로 Copy
AiM	MAC_SBK [opsize] [channel_mask] [bank] [row]	Single Bank에서 256b MAC 수행
AiM	MAC_ABK [opsize] [channel_mask] [row]	모든 Bank에서 256b MAC 수행

AiM Simulator CMDs

❑ Deep Learning CMDs

- Simulator의 CMD와 논문의 CMD는 구성이 약간 다르다
- Simulator의 CMDs

TABLE I NEWLY DEFINED DEDICATED DL CMD SET		
Type	CMD	Description
Bank Activation	ACT4, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks parallel
Compute	MACSB, MAC4B, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWMUL	Perform element-wise multiplication
Data	RDCP	Copy data from a bank to global buffer
	WRCP	Copy data from global buffer to a bank
	WRGB	Write to global buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

[GDDR6 based AiM paper의 Table]

유형	명령어	세부 사항
AiM	AF [channel_mask]	모든 Bank에서 Activation Func 수행
AiM	EWMUL [opsize] [channel_mask] [row]	4개 Bank에서 Elementwise Mult 수행
AiM	EWADD [opsize] [GPR_0] [GPR_1]	GPR을 사용한 Elementwise Add 수행
AiM	SYNC	ISR 명령어 간의 Barrier
AiM	EOC	계산 종료, 시뮬레이션 종료
W	MEM [channel_id] [bank] [row]	Bank에 대한 기존 Write 접근
R	MEM [channel_id] [bank] [row]	Bank로부터의 기존 Read 접근
W	GPR [GPR_0]	GPR에 256b Write
R	GPR [GPR_0]	GPR로부터 256b Read
W	CFR [CFR_0] [data]	Data를 이용해 CFR 구성

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
1. # AiM [opcode] [opsize] [GPR_id_0] [GPR_id_1] [channel_mask] [bank_id] [row_id]
2. # R/W GPR [GPR_id]
3. # R/W CFR "[CFR_id {0: broadcast 1: EWUL_bg 2: afm}]" [data]
4. # R/W MEM [channel_id] [bank_id] [row_id]
5. # Access granularity [GPR: BURST bytes] [CFR: 2 bytes] [MEM: Burst bytes] where BURST = 32 bytes
6.
7. # Regular write to row [8-11] in bank [2] of channels [0-3]
8. # R/W MEM [channel_id] [bank_id] [row_id]
9. W MEM 0 2 8
10. W MEM 1 2 9
11. W MEM 2 2 10
12. W MEM 3 2 11
13.
14. # Writing to GPR [0-1] for bank data
15. # R/W GPR [GPR_id]
16. W GPR 0
17. W GPR 1
18.
19. # Writing to GPR [2-3] for global buffer data
20. # R/W GPR [GPR_id]
21. W GPR 2
22. W GPR 3
23.
24. # Writing to GPR [4-7] for bias data
25. # R/W GPR [GPR_id]
26. W GPR 4
27. W GPR 5
28. W GPR 6
29. W GPR 7
30.
31. # "Write [2-data = 512 bits] from GPR[0 1] to channels [0-3] bank [0] row [0] (channel mask must only specify 1 channel)"
32. # AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]
33. AiM WR_SBK 0 1 0 0
34. AiM WR_SBK 0 2 0 0
35. AiM WR_SBK 0 4 0 0
36. AiM WR_SBK 0 8 0 0
37. AiM WR_SBK 1 1 0 0
38. AiM WR_SBK 1 2 0 0
39. AiM WR_SBK 1 4 0 0
40. AiM WR_SBK 1 8 0 0
41.
42. # "Write [2-data = 512 bits] from GPR[2 3] to channels [0-3] global buffers"
43. # AiM [opcode] [opsize] [GPR_id_0] [channel_mask]
44. AiM WR_GB 2 2 15
45.
46. # Write [4-data = 1024 bits] from GPR[4-7] to channels [0-3] * 16 MAC accumulators
47. # AiM [opcode] [GPR_id_0] [channel_mask]
48. AiM WR_BIAS 4 15
49.
```

- Simul이기에, Data는 따로 주고 받지 않고, Address만 이용합니다.

1. **Memory Write** - R/W MEM [channel_id] [bank_id] [row_id]

- W MEM 0 2 8 : Regular write to row[8] in bank[2] of channel[0]

2. **GPR Write** - R/W GPR [GPR_id]

- W GPR 0 : Writing 256bit to GPR[0] (GPR: General Purpose Register)

3. **BANK Write** - AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]

- AiM WR_SBK 0 1 0 0 : Write 256bit from GPR[0] to channel[0] bank[0] row[0]

4. **GB Write** - AiM [opcode] [opsize] [GPR_id_0] [channel_mask]

- AiM WR_GB 2 2 15 :

Write [2-data = 512 bits] from GPR[2-3] to channels [0-3] global buffers

- opsize는 가져올 data 크기를 조절
- opsize = 2이면, 256b data 2개 (= 512b) 가져옴
- GPR은 1개 당 256b 저장 가능
- 512b 가져오려면 GPR id: 2부터 시작해 GPR id: 3까지 이용해야 함
- channel mask: 15 = 32'b0000_1111이므로 channel 0~3을 의미

4. **16개 BANK의 16b MAC Reg에 Write** - AiM [opcode] [GPR_id_0] [channel_mask]

- AiM WR_BIAS 4 15 :

Write [4-data = 1024 bits] from GPR[4-7] to channels [0-3] * 16 MAC accumulators

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
50. # set CFR[mr13.broadcast at address 0x0010] to 1 (MAC from global buffer)
51. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
52. W CFR 0 1
53.
54. # MAC operation between 2 columns of all banks in channels [0-3] with 2 columns of global buffer
55. # AiM [opcode] [opsize] [channel_mask] [row_id]
56. AiM MAC_ABK 2 15 0
57.
58. # set CFR[mr13.broadcast at address 0x0010] to 0 (MAC from next bank)
59. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
60. W CFR 0 0
61.
62. # MAC operation between 2 columns of half of the banks in channels [0-3] with 2 columns of the next banks
63. # AiM [opcode] [opsize] [channel_mask] [row_id]
64. AiM MAC_ABK 2 15 0
65.
66. # set CFR[mr13.broadcast at address 0x0010] to 1 (MAC from global buffer)
67. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
68. W CFR 0 1
69.
70. # MAC operation between 2 columns of bank [0] in channels [0-3] with 2 columns of global buffer
71. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
72. AiM MAC_SBK 2 15 0 0
73.
74. # set CFR[mr13.broadcast at address 0x0010] to 0 (MAC from next bank)
75. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
76. W CFR 0 0
77.
78. # MAC operation between 2 columns of bank [0] in channels [0-3] with 2 columns of the next banks
79. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
80. AiM MAC_SBK 2 15 0 0
81.
82. # set CFR[mr14.ewmul_bg at address 0x0018] to 1 (EWMUL only 1 bankgroup)
83. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
84. W CFR 1 1
85.
86. # "ELMUL operation between 1 bank group 2 columns of channels [0-3] with 2 columns of the next banks"
87. # AiM [opcode] [opsize] [channel_mask] [row_id]
88. AiM EWMUL 2 15 0
89.
90. # set CFR[mr14.ewmul_bg at address 0x0018] to 0 (EWMUL in all bankgroups)
91. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
92. W CFR 1 0
93.
94. # "ELMUL operation between all bank groups 2 columns of channels [0-3] with 2 columns of the next banks"
95. # AiM [opcode] [opsize] [channel_mask] [row_id]
96. AiM EWMUL 2 15 0
97.
98. # set CFR[mr13.afm at address 0x001C] to 1 (AF number 1)
99. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
100. W CFR 2 1
```

5. **set CFR** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 0 1 : set CFR[0] to **1** (= MAC from **global buffer**)

6. **MAC Operation** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM MAC_ABK 2 15 0 :

MAC : (2 columns of all banks in channels [0-3]) x (2 columns of global buffer)

7. **set CFR** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 0 0 : set CFR[0] to **0** (= MAC from **next bank**)

8. **MAC Operation** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM MAC_ABK 2 15 0 :

MAC : (2 columns of half banks in channels [0-3]) x (2 columns of the next banks)

9.

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
101.
102. # AF operation in all banks of channels [0-3]
103. # AiM [opcode] [channel_mask]
104. AiM AF 15
105.
106. # Reading MAC accumulators of all banks of channels [0-3] to GPRs[8-11]
107. # AiM [opcode] [GPR_id_0] [channel_mask]
108. AiM RD_MAC 8 15
109.
110. # Reading AF results of all banks of channels [0-3] to GPRs[12-15]
111. # AiM [opcode] [GPR_id_0] [channel_mask]
112. AiM RD_AF 12 15
113.
114. # ELADD operation between GPRs[8-11] and GPRs[12-15]
115. # AiM [opcode] [opsize] [GPR_id_0] [GPR_id_1]
116. AiM EWADD 4 8 12
117.
118. # Copying 2 columns from bank 0 row 0 to global buffers in channels [0-3]
119. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
120. AiM COPY_BKGB 2 15 0 0
121.
122. # Copying 2 columns from global buffers to bank 0 row 0 in channels [0-3]
123. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
124. AiM COPY_GBBK 2 15 0 0
125.
126. # Read [2-data = 512 bits] from channels [0-3] bank [0] row [0] to GPR[20-27] (channel mask must only specify 1 channel)
127. # AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]
128. AiM RD_SBK 20 1 0 0
129. AiM RD_SBK 21 1 0 0
130. AiM RD_SBK 22 2 0 0
131. AiM RD_SBK 23 2 0 0
132. AiM RD_SBK 24 4 0 0
133. AiM RD_SBK 25 4 0 0
134. AiM RD_SBK 26 8 0 0
135. AiM RD_SBK 27 8 0 0
136.
137. # AiM [opcode] [GPR_id_0] [channel_mask] [row_id]
138. AiM WR_ABK 4 1 5
139. AiM WR_ABK 5 2 5
140. AiM WR_ABK 6 4 5
141. AiM WR_ABK 7 8 5
142.
143. # Finishing compute (MUST have this instruction at the end of the trace file)
144. # AiM [opcode]
145. AiM EOC
```

10.

AiM Simulator Main Function

Main Function의 초기화 및 설정 부분

```
1. int main(int argc, char *argv[]) {
2.     Ramulator::AiMInfo::init();
3.
4.     // Parse command line arguments
5.     argparse::ArgumentParser program("Ramulator", "2.0");
6.     program.add_argument("-c", "--config").metavar("\dumped YAML configuration").help("String dump of the yaml configuration.");
7.     program.add_argument("-f", "--config_file").metavar("path-to-configuration-file").help("Path to a YAML configuration file.");
8.     program.add_argument("-p", "--param").metavar("KEY=VALUE").help("Specify parameter to override in the configuration file. Repeat this option to change multiple parameters.");
9.     program.add_argument("-t", "--trace_file").metavar("path-to-trace-file").help("Path to the trace file.");
10.
11.     try {
12.         program.parse_args(argc, argv);
13.     } catch (const std::runtime_error &err) {
14.         spdlog::error(err.what());
15.         std::cerr << program;
16.         std::exit(1);
17.     }
18.
19.     // Are we accepting the configuration YAML through commandline dump?
20.     bool use_dumped_yaml = false;
21.     std::string dumped_config;
22.     if (auto arg = program.present<std::string>("-c")) {
23.         use_dumped_yaml = true;
24.         dumped_config = *arg;
25.     }
26.
27.     // Are we getting a path to a YAML document?
28.     bool use_yaml_file = false;
29.     std::string config_file_path;
30.     if (auto arg = program.present<std::string>("-f")) {
31.         use_yaml_file = true;
32.         config_file_path = *arg;
33.     }
34.
35.     // Are we overriding some parameters in a YAML document from the comand line?
36.     bool has_param_override = false;
37.     std::vector<std::string> params;
38.     if (auto arg = program.present<std::vector<std::string>>("-p")) {
39.         has_param_override = true;
40.         params = *arg;
41.     }
42.
43.     // Some sanity check of the inputs
44.     if (use_dumped_yaml && use_yaml_file) {
45.         spdlog::error("Dumped config and loaded config cannot be used together!");
46.         std::cerr << program;
47.         std::exit(1);
48.     } else if (!use_dumped_yaml || use_yaml_file) {
49.         spdlog::error("No configuration specified!");
50.         std::cerr << program;
51.         std::exit(1);
52.     }
53.
54.     if (use_dumped_yaml && has_param_override) {
55.         spdlog::warn("Using dumped configuration. Parameter overrides with -p/--param will be ignored!");
56.     }
57.
58.     // Parse the configurations
59.     YAML::Node config;
60.     if (use_dumped_yaml) {
61.         std::string dumped_config = program.get<std::string>("-c");
62.         config = YAML::Load(dumped_config);
63.     } else if (use_yaml_file) {
64.         config = Ramulator::Config::parse_config_file(config_file_path, params);
65.     }
66.
67.     if (auto arg = program.present<std::string>("-t")) {
68.         Ramulator::trace_file_path = *arg;
69.         Ramulator::use_trace_file_path = true;
70.     }
71.
72.     // Instantiate the frontend of the simulated system, this is one of the top-level objects in Ramulator 2.0.
73.     // It also recursively instantiate all components in the frontend.
74.     auto frontend = Ramulator::Factory::create_frontend(config);
75.     // Instantiate the memory system of the simulated system, this is one of the top-level objects in Ramulator 2.0
76.     // It also recursively instantiate all components in the memory system.
77.     auto memory_system = Ramulator::Factory::create_memory_system(config);
78.
79.     // Connect the frontend and the memory system together,
80.     // this recursively calls the "setup" function in all instantiated components
81.     // so that they can get each other's parameters (if needed) after their initialization
82.     frontend->connect_memory_system(memory_system);
83.     memory_system->connect_frontend(frontend);
84.
85.     // Get the relative clock ratio between the frontend and memory system
86.     int frontend_tick = frontend->get_clock_ratio();
87.     int mem_tick = memory_system->get_clock_ratio();
88.
89.     int tick_mult = frontend_tick * mem_tick;
```

1. 초기화 및 전역 상태 설정

- AiM ISR(Instruction Set Register) Info 초기화
- ISR 명령어 Mapping: String으로 들어오는 Trace Inst(ex. "WR_SBK" 등)를 Opcode enum으로 변환
- Type Mapping: Mem Access Region(GPR, CFR, MEM 등)의 정보를 Mapping

2. Command Line Parameters Parsing (ex. ./ramulator2 -f test/example.yaml -t test/all_isr.trace)

- -c / --config YAML 설정을 string으로 직접 전달 (dump 형식)
- -f / --config_file YAML 설정 파일 경로
- -p / --param 설정값 동적 override
- -t / --trace_file Instruction들이 포함된 Trace File 경로

3. 설정 File(ex. example.yaml) Parsing

4. Frontend Load 및 Memory System Load

- Yaml File에 주어진 Frontend Load 및 Memory System Load를 진행
 - ex. Frontend = AiMTrace / MemorySystem = AiMDRAM

request.h – init() 함수

```
210 public:
211     static void init() {
212         aim_opcode_to_str[Opcode::ISR_WB_SBK] = "ISR_WB_SBK";
213         opcode_str_to_aim_isr["ISR_WB_SBK"] = AIMISR(Opcode::ISR_WB_SBK,
214             (AIMISR::field::gpr_addr_0,
215             AIMISR::field::channel_mask,
216             AIMISR::field::bank_index,
217             AIMISR::field::row_addr),
218             false, // channel count req. mem
219             false, // ADR_PRR_blocking
220             false, // require reg. fu. wcd
221             "column" // target_level
222         );
223
224         aim_opcode_to_str[Opcode::ISR_WB_AKB] = "ISR_WB_AKB";
225         opcode_str_to_aim_isr["ISR_WB_AKB"] = AIMISR(Opcode::ISR_WB_AKB,
226             (AIMISR::field::gpr_addr_0,
227             AIMISR::field::channel_mask,
228             AIMISR::field::row_addr),
```

AiM Simulator Main Function

□ Main Function의 Simulation 실행 부분

```
91. for (uint64_t i = 0;; i++) {
92.     if (((i % tick_mult) % mem_tick) == 0) {
93.         frontend->tick();
94.     }
95.
96.     if (frontend->is_finished()) {
97.         break;
98.     }
99.
100.    if ((i % tick_mult) % frontend_tick == 0) {
101.        memory_system->tick();
102.    }
103. }
104.
105. // Finalize the simulation. Recursively print all statistics from all components
106. frontend->finalize();
107. memory_system->finalize();
108.
109. return 0;
110. }
111.
```

5. Simulation 실행 Loop

- Ramulator2.0와 동일하게 기본 설정을 한 후, tick기반 simulation을 실행

AiM Simulator Frontend Function

□ Frontend: aim_trace.cpp – tick()

```
1. void tick() override {
2.     if ((remaining_req == false) && (m_trace_reached_EOC == false)) {
3.         AiMTrace_request = get_host_request();
4.         remaining_req = true;
5.     }
6.     if (remaining_req == true) {
7.         bool request_sent = m_memory_system->send(AiMTrace_request);
8.         if (request_sent)
9.             remaining_req = false;
10.    }
11.};
```

request_sent = false가 되기 위해서는 아래의 조건을 만족해야 함

```
src > memory_system > impl > aim_DRAM_system.cpp > {} Ramulator > AiMDRAMSystem > send(Request)
14 namespace Ramulator {
15
16     #define ISR_SIZE (1 << 21)
17     #define MAX_CHANNEL_COUNT 32
18
19     ...
20
187 if (request_queue.size() == ISR_SIZE) {
188     s_ISR_queue_full++;
189     return false;
190 }
```

aim_trace.cpp 다른 함수들도 존재하지만 가장 중요한 tick()을 살펴본다.

▪ tick()

1. Memory System에 보내지 못한 Request가 없거나 trace가 EOC에 도달 X 시
 - Trace file에서 다음 Request를 가져옴
 - EOC = Trace의 종료 (= Simulation의 종료)를 나타내는 Request
2. Memory System에 Request를 보냄 (= trace file에서 가져온 Request)
 - m_memory_system->send(AiMTrace_request)가 false이라면,
 - 아래 ISR_SIZE 보다 request_queue.size()가 커야 함
 - send()에 실패할 경우 Request가 stall이 됨

▪ tick() 외 함수들

- init(): trace file 환경 초기화
- get_host_request(): Trace File의 Line들을 Decoding (= Instruction Decode)
- receive(): EOC 완료 callback (EOC command: simulation 완료를 trigger)
- is_finished(): Simulation 종료 판정
(receive()에서 return true시 is_finished()가 true가 되는 흐름으로 진행)

AiM Simulator Frontend -> Memory System Function

❑ MemorySystem: aim_dram_system.cpp – send()

```
1. bool send(Request req) override {
2.
3.     ///// - debug by MSKIM in ISL yonsei
4.     m_logger->debug("----- Cycle: {}, Get Host Request: {} -----", m_clk, req.str());
5.     /////
6.
7.     if (request_queue.size() == ISR_SIZE) {
8.         s_ISR_queue_full++;
9.         ///// - debug by MSKIM in ISL yonsei
10.        m_logger->debug("== Cycle: {}, ISR queue full for request: {} ==", m_clk, req.str());
11.        /////
12.        return false;
13.    }
14.    request_queue.push(req);
15.    // m_logger->info("[CLK {}] {} pushed to the queue!", m_clk, req.str());
16.
17.    switch (req.type) {
18.    case Type::AIM: {
19.        s_num_AiM_requests[req.opcode]++;
20.        break;
21.    }
22.    case Type::Read:
23.    case Type::Write: {
24.        s_num_RW_requests[req.type][req.mem_access_region]++;
25.        break;
26.    }
27.    default: {
28.        throw ConfigurationError("AiMDRAMSystem: unknown request type {}!", (int)req.type);
29.        break;
30.    }
31.    }
32.
33.    return true;
34. }
```

CMD 통계는 아래와 같이 report가 출력 됨

```
total_num_AiM_ISR_COPY_GBBK_requests: 1 # total number of AiM ISR_COPY_GBBK requests
total_num_W_MEM_requests: 4 # total number of W MEM requests
total_num_AiM_ISR_WR_SBK_requests: 8 # total number of AiM ISR_WR_SBK requests
total_num_R_GPR_requests: 0 # total number of R GPR requests
total_num_AiM_ISR_WR_BIAS_requests: 1 # total number of AiM ISR_WR_BIAS requests
```

aim_trace.cpp – tick()에서 m_memory_system->send() 호출하므로 살펴본다.

▪ send()

1. request_queue에 자리가 있는 지 확인 후, request를 queue에 저장

- $ISR_SIZE = 1 \ll 21 = 2,097,152$
- queue 저장 거절 시 → 다음 cycle에 재시도
- request_queue에 저장된 req는 m_memory_system – tick()에서 처리

2. Request Type별 통계 계산

- Type::AIM 인 CMD 통계 update

CMD 예시	Statistic Update되는 배열
ISR_WR_SBK	s_num_AiM_requests[Opcode::ISR_WR_SBK]
ISR_MAC_ABK	s_num_AiM_requests[Opcode::ISR_MAC_ABK]
ISR_EOC	s_num_AiM_requests[Opcode::ISR_EOC]

- Type::Read 및 Type::Write인 CMD 통계 update

```
// 예: GPR에서 읽은 요청 개수
s_num_RW_requests[Type::Read][MemAccessRegion::GPR]++;

// 예: CFR에 쓴 요청 개수
s_num_RW_requests[Type::Write][MemAccessRegion::CFR]++;

// 예: MEM에서 읽은 요청 개수
s_num_RW_requests[Type::Read][MemAccessRegion::MEM]++;
```

AiM Simulator PIM Structure

□ base/request.h

```
1. // Basic request id convention
2. // 0 = Read, 1 = Write. The device spec defines all others
3. enum class Type {
4.     Read = 0,
5.     Write = 1,
6.     RefAllBank = 2,
7.     RefSingleBank = 3,
8.     AIM = 4,
9.     MAX = 5
10. };
11.
12. enum class MemAccessRegion {
13.     MIN,
14.     GPR,
15.     CFR,
16.     MEM,
17.     MAX
18. };
19.
20. // ISR Command Opcodes
21. enum class Opcode {
22.     MIN = 0,
23.     ISR_WR_SBK = 1, // Write [op_size * 256 bits] from [GPR * 32] to [a single bank] of channel [#channel_address]
24.     ISR_WR_GB = 2, // Write the same [op_size * 256 bits] starting from [GPR * 32]
25.                     // to the [Global Buffer] of [channel_mask] channels
26.                     // [NOT_IMPLEMENTED] source could also be [host]
27.     ISR_WR_BIAS = 3, // Write [opsize * 16 x 16-bits] bits from [GPR * 32]
28.                     // to [MAC accumulator of 16 banks] of [opsize or channel_mask] channels
29.     ISR_WR_AFLUT = 4, // Write activation function data from [host] to [?]
30.     ISR_RD_MAC = 5, // Read data from [MAC accumulator of all banks] to [GPR]
31.     ISR_RD_AF = 6, // Read data from [AF results of all banks] to [GPR]
32.     ISR_RD_SBK = 7, // [NOT_LISTED] Read data from [a single bank] to [host]
33.     ISR_COPY_BKGB = 8, // Copy data from [a single bank] to [the Global Buffer]
34.     ISR_COPY_GBBK = 9, // Copy data from [the Global Buffer] to [a single bank]
35.     ISR_MAC_SBK = 10, // Perform MAC operation between [a single bank] and [Global Buffer]
36.     ISR_MAC_ABK = 11, // Perform MAC operation between [all banks] and [Global Buffer]
37.     ISR_AF = 12, // Perform Activation Function operation on [all banks]
38.     ISR_EWMUL = 13, // Element wise multiplication between 2 banks of 1 or all bank group(s)
39.     ISR_EWADD = 14, // Element wise multiplication between 2 GPR addresses
40.     ISR_WR_ABK = 15, // Write [16 x 16 bits] from [GPR * 32] to [16 banks] of channel [#channel_address]
41.     ISR_EOC = 16, // End of compute for the current kernel
42.     ISR_SYNC = 17,
43.     MAX = 18
44.     // ISR_WR_HBK, // [NOT_IMPLEMENTED] Write data from [GPR] to [8 banks]
45.     // ISR_WR_ABK, // [NOT_IMPLEMENTED] Write data from [GPR] to [all banks]
46.     // ISR_WR_GPR, // [NOT_IMPLEMENTED] Write data from [host] to [GPR]
47.     // ISR_MAC_HBK, // [NOT_IMPLEMENTED] Perform MAC operation between [8 banks] and [Global Buffer]
48. };
```

본격적으로 Memory System을 살피기 전에, AiM의 CMD가 어떻게 구성되어 있는지 확인

1. Request의 Type을 알려줌. (가장 큰 범위라고 생각하면 됨)

- Read: Normal DRAM Read Operation Request
- Write: Normal DRAM Write Operation Request
- AIM: DRAM Processing in Memory Request
- RefAllBank, RefSingleBank: Not Used (선언만 되어 있음. 동작 구현 없음)

2. 해당 Request가 어느 Memory Region에 Access하는지를 알려줌

- GPR: General Purpose Registers
- CFR: Configuration Registers
- MEM: Memory(DRAM Bank)
- ex. AiM WR_SBK 0 1 0 0 : Write 256bit from GPR[0] to channel[0] bank[0] row[0]
즉, Type = Write, MemAccessRegion = MEM이 됨 (Memory에 Write를 하므로)

3. PIM Commands의 Opcodes를 enum 형식으로 정리

AiM Simulator PIM Structure

□ base/request.h

```
1. struct Request {
2.     Addr_t addr = -1;
3.     Data_t data;
4.     AddrVec_t addr_vec{};
5.     int host_req_id = -1;
6.     int AiM_req_id = -1;
7.
8.     Type type = Type::MAX;
9.
10.    MemAccessRegion mem_access_region = MemAccessRegion::MAX;
11.
12.    Opcode opcode = Opcode::MAX;
13.
14.    // [NOT_IMPLEMENTED] Increment order for ISR_WR_SBK, ISR_WR_HBK, and ISR_WR_ABK operations
15.    // struct IncrementOrder {
16.    //     enum : int {
17.    //         IO_ROW_COL, // {ROW_ADDR[13:0], COL_ADDR[5:0]}++
18.    //         IO_BK_COL,  // {BK_ADDR[3:0], COL_ADDR[5:0]}++
19.    //         IO_COL_CH,  // {COL_ADDR[3:0], BK_ADDR[5:0]}++
20.    //         MAX
21.    //     };
22.    // };
23.
24.    int32_t opsize = -1;
25.
26.    // [NOT_IMPLEMENTED] Source of ISR_WR_GB is host (false) or GPR (true)
27.    // bool use_GPR = false;
28.
29.    // GPR address 0 USED for: ISR_WR_SBK, ISR_WR_ABK, ISR_WR_GB, ISR_WR_BIAS, ISR_RD_MAC, ISR_EWADD
30.    // GPR address 1 USED for: ISR_EWADD
31.    Addr_t GPR_addr_0 = -1;
32.    Addr_t GPR_addr_1 = -1;
33.
34.    // This request will be broadcasted/multicast to the channels
35.    // whose bit is set in channel mask.
36.    // NOT USED in ISR_EWADD.
37.    // Channel mask must show 1 channel in ISR_WR_ABK ISR
38.    int64_t channel_mask = -1;
39.
```

```
40.    // This request will be sent to a specific bank. USED only in single-bank ISRs, i.e.,
41.    // ISR_WR_SBK, ISR_RD_SBK, ISR_COPY_BKGB, ISR_COPY_GBBK, ISR_MAC_SBK, and ISR_EWMUL
42.    int16_t bank_index = -1;
43.
44.    // Bank row and column address, NOT USED for the operations without DRAM bank source/dst:
45.    // ISR_WR_BIAS, ISR_WR_AFLUT, ISR_RD_MAC, ISR_RD_AF, ISR_AF, and ISR_EWADD
46.    // In addition, row_addr is NOT USED for ISR_WR_GB
47.    int32_t row_addr = -1;
48.    int32_t col_addr = -1;
49.
50.    // Thread (register) index (0 or 1) for MAC and AF results
51.    int8_t thread_index = -1;
52.
53.    // Broadcast only USED for ISR_MAC_SBK and ISR_MAC_ABK
54.    // vector data for MAC is from GB (0) or next bank (1)
55.    int16_t broadcast = -1;
56.
57.    // AFM only USED for ISR_AF
58.    // Activation Function mode selects AF (0-7)
59.    int16_t afm = -1;
60.
61.    // ewmul bank group only USED for ISR_MAC_ABK
62.    // EWMUL in one bank group (0) or all bank groups (1)
63.    int16_t ewmul_bg = -1;
64.
65.    int source_id = -1; // An identifier for where the request is coming from (e.g., which core)
66.
67.    int command = -1; // The command that need to be issued to progress the request
68.    int final_command = -1; // The final command that is needed to finish the request
69.
70.    Clk_t arrive = -1; // Clock cycle when the request arrive at the memory controller
71.    Clk_t issue = -1; // Clock cycle when the request issued at the memory controller
72.    Clk_t depart = -1; // Clock cycle when the request depart the memory controller
73.
74.    int type_id = -1;
75.
76.    std::function<void(Request &)> callback;
77.
78.    Request(Addr_t addr, int type_id);
79.    Request(AddrVec_t addr_vec, int type_id);
80.    Request(Addr_t addr, int type_id, int source_id, std::function<void(Request &)> callback);
81.    // Request(Request const &req);
82.
83.    std::string str();
84.
85.    bool is_reader();
86.};
```

- 다음 page에 절반씩 나눠서 설명

AiM Simulator PIM Structure

□ base/request.h

```
1. struct Request {
2.     Addr_t addr = -1;
3.     Data_t data;
4.     AddrVec_t addr_vec{};
5.     int host_req_id = -1;
6.     int AiM_req_id = -1;
7.
8.     Type type = Type::MAX;
9.
10.    MemAccessRegion mem_access_region = MemAccessRegion::MAX;
11.
12.    Opcode opcode = Opcode::MAX;
13.
14.    // [NOT_IMPLEMENTED] Increment order for ISR_WR_SBK, ISR_WR_HBK, and ISR_WR_ABK operations
15.    // struct IncrementOrder {
16.    //     enum : int {
17.    //         IO_ROW_COL, // {ROW_ADDR[13:0], COL_ADDR[5:0]}++
18.    //         IO_BK_COL,  // {BK_ADDR[3:0], COL_ADDR[5:0]}++
19.    //         IO_COL_CH,  // {COL_ADDR[3:0], BK_ADDR[5:0]}++
20.    //         MAX
21.    //     };
22.    // };
23.
24.    int32_t opsize = -1;
25.
26.    // [NOT_IMPLEMENTED] Source of ISR_WR_GB is host (false) or GPR (true)
27.    // bool use_GPR = false;
28.
29.    // GPR address 0 USED for: ISR_WR_SBK, ISR_WR_ABK, ISR_WR_GB, ISR_WR_BIAS, ISR_RD_MAC, ISR_EWADD
30.    // GPR address 1 USED for: ISR_EWADD
31.    Addr_t GPR_addr_0 = -1;
32.    Addr_t GPR_addr_1 = -1;
33.
34.    // This request will be broadcasted/multicasted to the channels
35.    // whose bit is set in channel mask.
36.    // NOT USED in ISR_EWADD.
37.    // Channel mask must show 1 channel in ISR_WR_ABK ISR
38.    int64_t channel_mask = -1;
39.}
```

- struct Request: 실제 Memory Request 작업 Instance
- addr
- data
- addr_vec{}
- host_req_id
- AiM_req_id
- type
- mem_access_region
- opcode
- opsize
- GPR_addr_0
- GPR_addr_1
- channel_mask

AiM Simulator PIM Structure

❏ base/request.h

```
40. // This request will be sent to a specific bank. USED only in single-bank ISRs, i.e.,
41. // ISR_WR_SBK, ISR_RD_SBK, ISR_COPY_BKGB, ISR_COPY_GBBK, ISR_MAC_SBK, and ISR_EWMUL
42. int16_t bank_index = -1;
43.
44. // Bank row and column address, NOT USED for the operations without DRAM bank source/dst:
45. // ISR_WR_BIAS, ISR_WR_AFLUT, ISR_RD_MAC, ISR_RD_AF, ISR_AF, and ISR_EWADD
46. // In addition, row_addr is NOT USED for ISR_WR_GB
47. int32_t row_addr = -1;
48. int32_t col_addr = -1;
49.
50. // Thread (register) index (0 or 1) for MAC and AF results
51. int8_t thread_index = -1;
52.
53. // Broadcast only USED for ISR_MAC_SBK and ISR_MAC_ABK
54. // vector data for MAC is from GB (0) or next bank (1)
55. int16_t broadcast = -1;
56.
57. // AFM only USED for ISR_AF
58. // Activation Function mode selects AF (0-7)
59. int16_t afm = -1;
60.
61. // ewmul bank group only USED for ISR_MAC_ABK
62. // EWMUL in one bank group (0) or all bank groups (1)
63. int16_t ewmul_bg = -1;
64.
65. int source_id = -1; // An identifier for where the request is coming from (e.g., which core)
66.
67. int command = -1; // The command that need to be issued to progress the request
68. int final_command = -1; // The final command that is needed to finish the request
69.
70. Clk_t arrive = -1; // Clock cycle when the request arrive at the memory controller
71. Clk_t issue = -1; // Clock cycle when the request issued at the memory controller
72. Clk_t depart = -1; // Clock cycle when the request depart the memory controller
73.
74. int type_id = -1;
75.
76. std::function<void(Request &)> callback;
77.
78. Request(Addr_t addr, int type_id);
79. Request(AddrVec_t addr_vec, int type_id);
80. Request(Addr_t addr, int type_id, int source_id, std::function<void(Request &)> callback);
81. // Request(Request const &req);
82.
83. std::string str();
84.
85. bool is_reader();
86. };
```

- bank_index
- row_addr
- col_addr
- thread_index
- Broadcast
- afm
- ewmul_bg
- source_id
- command
- final_command
- arrive
- issue
- depart
- type_id
- is_reader()

AiM Simulator PIM Structure

□ base/request.h

```
1. class AiMISR {
2.
3. public:
4.     enum class Field {
5.         opcode,
6.         GPR_addr_0,
7.         GPR_addr_1,
8.         channel_mask,
9.         bank_index,
10.        row_addr,
11.        col_addr,
12.        thread_index
13.    };
14.
15.    std::map<Field, std::string> field_to_str = {
16.        {Field::opcode, "opcode"},
17.        {Field::GPR_addr_0, "GPR_addr_0"},
18.        {Field::GPR_addr_1, "GPR_addr_1"},
19.        {Field::channel_mask, "channel_mask"},
20.        {Field::bank_index, "bank_index"},
21.        {Field::row_addr, "row_addr"},
22.        {Field::col_addr, "col_addr"},
23.        {Field::thread_index, "thread_index"},
24.    };
25.
26.    AiMISR(Opcode opcode_ = Opcode::MAX,
27.           std::vector<Field> legal_fields_ = {},
28.           bool channel_count_eq_one_ = false,
29.           bool AiM_DMA_blocking_ = false,
30.           bool require_reg_RW_mod_ = false,
31.           std::string target_level_ = "")
32.        : opcode(opcode_),
33.          legal_fields(legal_fields_),
34.          channel_count_eq_one(channel_count_eq_one_),
35.          AiM_DMA_blocking(AiM_DMA_blocking_),
36.          require_reg_RW_mod(require_reg_RW_mod_),
37.          target_level(target_level_) {}
38.
39.    Opcode opcode;
40.
41.    std::vector<Field> legal_fields;
42.
43.    bool channel_count_eq_one;
44.    bool AiM_DMA_blocking;
45.    bool require_reg_RW_mod;
46.
47.    std::string target_level;
48.
49.    bool is_field_legal(Field field) {
50.        if (std::count(legal_fields.begin(), legal_fields.end(), field))
51.            return true;
52.        return false;
53.    }
54.
55.    template <typename T>
56.    void is_field_value_legal(Field field, T value);
57. };
58.
59.
```

class AiMISR: ISR Command 정의 - 각 ISR opcode가 어떤 구조/특성을 가져야 하는지 정의

abcd

AiM Simulator PIM Structure

□ base/request.h

```
1. class AiMISRInfo {
2. private:
3.     static std::map<std::string, AiMISR> opcode_str_to_aim_ISR;
4.     static std::map<Opcode, std::string> aim_opcode_to_str;
5.
6.     static std::map<std::string, Type> str_to_type;
7.     static std::map<Type, std::string> type_to_str;
8.
9.     static std::map<std::string, MemAccessRegion> str_to_mem_access_region;
10.    static std::map<MemAccessRegion, std::string> mem_access_region_to_str;
11.
12. public:
13.     static void init() {
14.         aim_opcode_to_str[Opcode::ISR_WR_SBK] = "ISR_WR_SBK";
15.         opcode_str_to_aim_ISR["ISR_WR_SBK"] = AiMISR(Opcode::ISR_WR_SBK,
16.             {AiMISR::Field::GPR_addr_0,
17.              AiMISR::Field::channel_mask,
18.              AiMISR::Field::bank_index,
19.              AiMISR::Field::row_addr},
20.             false, // channel_count_eq_one
21.             false, // AiM_DMA_blocking
22.             false, // require_reg_RW_mod
23.             "column" // target_level
24.         );
25.
26.         aim_opcode_to_str[Opcode::ISR_WR_ABK] = "ISR_WR_ABK";
27.         opcode_str_to_aim_ISR["ISR_WR_ABK"] = AiMISR(Opcode::ISR_WR_ABK,
28.             {AiMISR::Field::GPR_addr_0,
29.              AiMISR::Field::channel_mask,
30.              AiMISR::Field::row_addr},
31.             true, // channel_count_eq_one
32.             false, // AiM_DMA_blocking
33.             false, // require_reg_RW_mod
34.             "column" // target_level
35.         );
36.
37.         aim_opcode_to_str[Opcode::ISR_WR_GB] = "ISR_WR_GB";
38.         opcode_str_to_aim_ISR["ISR_WR_GB"] = AiMISR(Opcode::ISR_WR_GB,
39.             {AiMISR::Field::opsize,
40.              AiMISR::Field::GPR_addr_0,
```

class AiMISRInfo: 모든 ISR 정의를 모아둔 Database - 모든 가능한 ISR 정의를 등록하고 관리

AiM Simulator Frontend Function

❑ **MemorySystem: aim_dram_system.cpp – tick()**

abcd